راهنمای کامل MVP در TouchGFX همراه با FreeRTOS

از فشردن کلید روی برد (STM32H7 + LTDC + SDRAM) تا نمایش متن روی LCD — شرح کامل منطق، توابع و آرگومان‌ها

مقدمه: هدف این سند

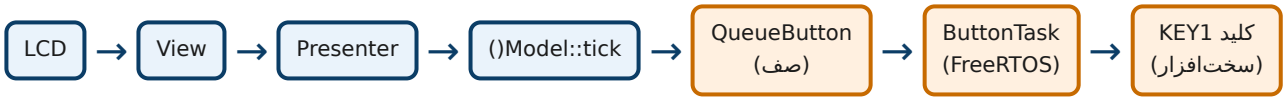
در پروژه شما یک کلید فیزیکی (`KEY1`) به پایه `PH` متصل است و وقتی فشرده می‌شود، متن «Pressed» یا «Not Pressed» روی LCD نمایش داده می‌شود. این مثال کوچک، تمام اجزای لازم برای هر برنامه‌ی واقعی TouchGFX را دارد: یک **تسک FreeRTOS** که سخت‌افزار را می‌خواند، یک **صف (Queue)** برای انتقال امن داده بین تسک‌ها، و معماری **MVP** برای رساندن داده به صفحه‌ی گرافیکی.

پس از مطالعه این سند باید بتوانید هر داده‌ای (دما، ولتاژ ADC، وضعیت رله، پیام UART و...) را به UI برسانید، و برعکس، هر رویداد UI (لمس دکمه، اسلایدر) را به سخت‌افزار برگردانید.

فهرست مطالب

1. منطق MVP و چرایی وجود آن
2. چرا صف FreeRTOS لازم است؟ (مسئله‌ی چند تسکی)
3. ساخت صف و تسک در CubeMX و کد تولیدشده
4. تسک دکمه (`StartButtonTask`) و تابع `osMessageQueuePut`
5. فایل‌های `Model.hpp` و `Model.cpp` و تابع `osMessageQueueGet`
6. `ModelListener` و `screenPresenter`
7. Wildcard، `screenView` و تابع‌های `Unicode`
8. جریان کامل داده، خطاهای رایج این پروژه، الگوی تعمیم و منابع

نقشه‌ی کلی مسیر داده



جعبه‌های نارنجی «دنیای RTOS و سخت‌افزار» هستند و جعبه‌های آبی «دنیای TouchGFX». تنها نقطه‌ی اتصال این دو دنیا، **صف** و **Model** است.

۱. منطق MVP (Model – View – Presenter)

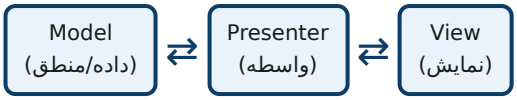
۱.۱ مسئله‌ای که MVP حل می‌کند

اگر همه‌چیز را در یک کلاس بنویسید (خواندن کلید، منطق برنامه، چیدمان گرافیکی)، با بزرگ شدن پروژه کد غیرقابل نگهداری می‌شود: تغییر ظاهر صفحه منطق را خراب می‌کند و برای تست منطق، باید LCD داشته باشید. MVP کد را به سه نقش مستقل تقسیم می‌کند:

نقش	مسئولیت	در پروژه‌ی شما
Model	نگهداری داده و منطق برنامه و ارتباط با دنیای بیرون (سخت‌افزار، RTOS). هیچ چیز از ظاهر گرافیکی نمی‌داند.	`Model.hpp / Model.cpp`
View	فقط نمایش: ویجت‌ها (Button، TextArea، Image...) و به‌روزرسانی آن‌ها. هیچ منطقی ندارد و فقط فرمان می‌گیرد.	`screenView + screenViewBase`
Presenter	واسطه: داده را از Model می‌گیرد، تصمیم می‌گیرد چه چیزی نشان داده شود، و به View فرمان می‌دهد. برعکس هم: رویداد View را به Model می‌رساند.	`screenPresenter + ModelListener`

۱.۲ قانون طلایی ارتباطات

- **View هرگز مستقیماً به Model دسترسی ندارد.** فقط به Presenter (از طریق اشاره‌گر `presenter`) دسترسی دارد.
- **Model هرگز View را نمی‌شناسد.** فقط یک «شنونده» (`ModelListener*`) را می‌شناسد که در واقع همان Presenter است.
- **Presenter** هم به View (با ارجاع `view`) و هم به Model (با اشاره‌گر `model` از ModelListener) دسترسی دارد.



جهت‌ها: رویداد کاربر (مثلاً لمس) از **View → Presenter → Model** می‌رود (با فراخوانی تابع). داده‌ی جدید از **Model → Presenter → View** می‌رود: Model تابع `notifyXxx()` را روی ModelListener صدا می‌زند، Presenter آن را override کرده و تابعی از View را فرا می‌خواند.

۱.۳ چرا ModelListener؟ (وارونگی وابستگی)

Model نباید به کلاس `screenPresenter` وابسته باشد، چون هر صفحه Presenter خودش را دارد و Model برای همه‌ی صفحات یکی است. راه‌حل: کلاس پایه‌ی `ModelListener` با توابع *virtual* خالی. هر Presenter از آن ارث می‌برد و فقط توابعی را که به آن‌ها علاقه دارد override می‌کند. Model همیشه روی همان یک اشاره‌گر پایه کار می‌کند؛ هر صفحه‌ی فعال، Presenter خودش را به Model متصل می‌کند و با عوض شدن صفحه، Presenter جدید جایگزین می‌شود (به همین دلیل در Model همیشه بررسی می‌کنیم `if (modelListener)` غیر تهی باشد).

۱.۴ چرخه‌ی عمر (Lifecycle) در TouchGFX

هنگام رفتن به یک صفحه، فریم‌ورک (در `FrontendApplicationBase` که خودکار تولید می‌شود) به ترتیب: View و Presenter را می‌سازد، Presenter را به Model وصل می‌کند (`model.bind(presenter)`)، و Presenter و View را به هم وصل می‌کند. سپس `setupScreen()` برای View و `activate()` برای Presenter صدا زده می‌شود. هنگام ترک صفحه: `deactivate()` و `tearDownScreen()` و سپس نابودی هر دو. این‌ها حافظه‌ی استاتیک (heap مخصوص TouchGFX) هستند، پس هیچ `new` و `delete` دستی لازم نیست.

تابع	زمان فراخوانی و کاربرد
`Model::Model()`	یک بار در شروع برنامه. مقدار اولیه‌ی `modelListener(0)` را می‌گذارد.
`Model::bind(listener)`	هنگام ساخت هر صفحه توسط فریم‌ورک: ذخیره‌ی Presenter فعلی در Model.
`Model::tick()`	در هر فریم GUI (معمولاً ۶۰ بار در ثانیه، هر ≈۱۶٫۷ms) از حلقه‌ی TouchGFX. محل مناسب برای خواندن از صف.
`View::setupScreen()`	موقع ورود به صفحه. مقداردهی اولیه‌ی ویجت‌ها. حتماً `screenViewBase::setupScreen()` را هم صدا بزنید.
`Presenter::activate()`	پس از ساخت صفحه. جای مناسب برای خواندن مقدار فعلی از Model و نمایش اولیه.
`Presenter::deactivate()`	موقع خروج از صفحه. پاک‌سازی.
`View::tearDownScreen()`	آخرین مرحله‌ی خروج از صفحه.

> **نکته‌ی مهم:** فایل‌های داخل پوشه‌ی `gui_generated` (مثل `screenViewBase.cpp`) را هرگز دستی ویرایش نکنید؛ با هر Generate Code بازنویسی می‌شوند. کد خود را فقط در پوشه‌های `gui/...` بنویسید.

۲. چرا صف FreeRTOS لازم است؟

۲.۱ دو دنیا، دو تسک

در پروژه‌ی شما سه تسک FreeRTOS ساخته شده است:

تسک	پشته (بایت)	اولویت	وظیفه
`defaultTask`	`128×4 = 512`	Normal	تسک پیش‌فرض CubeMX (خالی)
`GUI (TouchGFX_Task)`	`2048×4 = 8192`	Low	اجرای تمام TouchGFX: رندر، رویدادها، `Model::tick()`
`ButtonTask`	`128×4 = 512`	Low	خواندن دوره‌ای کلید

کد Model، Presenter و View **همگی داخل تسک GUI** اجرا می‌شوند. اما `StartButtonTask` در تسک دیگری اجرا می‌شود. دو تسک مستقل، هر لحظه ممکن است توسط scheduler قطع و جابه‌جا شوند.

۲.۲ چرا ButtonTask نباید مستقیم View را تغییر دهد؟

- کتابخانه‌ی TouchGFX **thread-safe نیست**: همه‌ی ویجت‌ها، بافرها و عملیات invalidate باید فقط از تسک GUI صدا زده شوند. اگر ButtonTask وسط رندر، بافر متن را تغییر دهد، نتیجه ممکن است خرابی تصویر یا کرش باشد.

- ButtonTask نباید به کلاس‌های C++ مربوط به صفحه (که ممکن است در همان لحظه نابود شوند، مثلاً هنگام تغییر صفحه) اشاره‌گر نگه دارد.
- جداسازی: بخش سخت‌افزار (C) و بخش گرافیک (C++) به هم وابسته نمی‌شوند.

۲.۳ راه‌حل: صف پیام (Message Queue)

صف، یک بافر FIFO است که FreeRTOS مدیریت می‌کند و **داخلش قفل‌گذاری (thread-safe) انجام شده است**. ButtonTask پیام را می‌*گذارد* (Put) و تسک GUI در `Model::tick()` آن را *برمی‌دارد* (Get). هیچ‌کدام منتظر دیگری نمی‌ماند (در حالت timeout = 0) و هیچ داده‌ی مشترکی بدون محافظت وجود ندارد.



۲.۴ چرا «در CubeMX» تعریف می‌کنیم؟

می‌توانستید صف را دستی در `main.c` هم بسازید؛ اما CubeMX سه فایده دارد: (۱) کد ساخت صف را در جای درست و قبل از شروع scheduler می‌نویسد، (۲) یک handle سراسری (`QueueButtonHandle`) و ساختار attributes می‌سازد که در تمام پروژه قابل استفاده است، (۳) در پروژه‌ی گروهی همه می‌بینند چه منابعی وجود دارد. بعداً برای استفاده در `Model.cpp` باید این handle را با `extern` اعلام کنیم تا لینکر بداند همان متغیر است.

ترتیب منطقی نوشتن: ۱) صف و تسک در CubeMX → ۲) فرستنده (ButtonTask) → ۳) گیرنده (Model::tick) → ۴) تابع notify در ModelListener → ۵) Override در Presenter → ۶) تابع نمایش در View → ۷) ویجت Wildcard در Designer.

۳. ساخت صف و تسک در CubeMX و کد تولیدشده

۳.۱ مراحل در محیط CubeMX

1. در بخش Interface، **Middleware → FREERTOS** را روی `CMSIS_V2` بگذارید.
2. تب **Tasks and Queues → Queues → Add**: نام `QueueButton`، طول (Queue Size) = 16، اندازه‌ی هر آیتم (Item Size) = `uint32_t` (۴ بایت)، Allocation = Dynamic.
3. تب **Tasks** → Add: تسک `ButtonTask` با Entry Function = `StartButtonTask`، اولویت و اندازه‌ی پشته مناسب.
4. پایه‌ی `KEY1` را به‌عنوان GPIO_Input با User Label برابر `KEY1` تنظیم کنید تا ماکروهای `KEY1_Pin` و `KEY1_GPIO_Port` ساخته شوند. بسته به مدار برد، Pull-up یا Pull-down مناسب را انتخاب کنید.
5. Generate Code.

خطای واقعی در پروژه‌ی شما: در `main.c` صف با `sizeof(uint16_t)` ساخته شده بود، ولی تسک متغیر `uint32_t` می‌فرستاد. در CubeMX، Item Size باید با نوع واقعی داده یکی باشد، وگرنه فقط بخشی از بایت‌ها کپی می‌شود. اگر فایل را دستی عوض کنید و دوباره Generate بزنید، CubeMX مقدار تنظیم‌شده خودش را بازمی‌نویسد، پس تنظیم را در CubeMX درست کنید.

۳.۲ کد تولیدشده در main.c

```
/* handle سراسری */
osMessageQueueId_t QueueButtonHandle;
const osMessageQueueAttr_t QueueButton_attributes = { .name = "QueueButton" };

/* داخل main() — قبل از osKernelStart */
osKernelInitialize();
QueueButtonHandle = osMessageQueueNew(16, sizeof(uint32_t), &QueueButton_attributes);
ButtonTaskHandle  = osThreadNew(StartButtonTask, NULL, &ButtonTask_attributes);
osKernelStart();   /* کنترل را می‌گیرد scheduler از اینجا به بعد */
```

۳.۳ تابع osMessageQueueNew و آرگومان‌ها

آرگومان	معنی	مقدار در پروژه
`uint32_t msg_count`	حداکثر تعداد پیام‌هایی که صف هم‌زمان نگه می‌دارد (عمق).	16
`uint32_t msg_size`	اندازه‌ی هر پیام بر حسب بایت. پیام **کپی** می‌شود (نه اشاره‌گر). باید برابر `sizeof` نوع فرستاده‌شده و خوانده‌شده باشد.	`sizeof(uint32_t)`
`const osMessageQueueAttr_t *attr`	ویژگی‌ها: نام صف، حافظه‌ی استاتیک (cb_mem/mq_mem) در صورت نیاز؛ NULL یعنی پیش‌فرض.	`&QueueButton_attributes`
مقدار بازگشتی	شناسه‌ی صف (`osMessageQueueId_t`) یا NULL در صورت شکست (مثلاً کمبود heap).	`QueueButtonHandle`

> حجم حافظه‌ی لازم از heap فریم‌ورک = `msg_count × msg_size` + ساختار کنترلی. اگر `configTOTAL_HEAP_SIZE` کم باشد، `osMessageQueueNew` مقدار NULL برمی‌گرداند و بعداً Put/Get بی‌صدا شکست می‌خورند. حتماً در برنامه‌ی نهایی NULL بودن handle را بررسی کنید.

۳.۴ تابع osThreadNew و آرگومان‌ها

آرگومان	معنی
`osThreadFunc_t func`	تابع ورودی تسک، با امضای `void f(void *argument)`. باید هرگز return نکند (حلقه‌ی بی‌نهایت).
`void *argument`	آرگومانی که به تابع تسک داده می‌شود (در اینجا NULL).
`const osThreadAttr_t *attr`	ساختار شامل `.stack_size`، `.name` (بایت)، `.priority` و ... .

نکته: `__weak` جلوی `StartButtonTask` یعنی اگر در فایل دیگری همین تابع بدون `__weak` تعریف شود، آن نسخه جایگزین می‌شود. در پروژه‌ی شما فقط یک تعریف وجود دارد، پس همین اجرا می‌شود.

۴. تسک دکمه: StartButtonTask

۴.۱ کد

```
void StartButtonTask(void *argument)
{
  uint32_t last = 0xFF;                /* مقدار غیرممکن تا اولین نمونه حتماً ارسال شود */
  for(;;)
  {
    uint32_t state = HAL_GPIO_ReadPin(KEY1_GPIO_Port, KEY1_Pin);

    if (state != last)                 /* فقط هنگام تغییر وضعیت */
    {
      last = state;
      osMessageQueuePut(QueueButtonHandle, &state, 0, 0);
    }
    osDelay(20);                       /* 20 میلی‌ثانیه خواب */
  }
}
```

نسخه‌ی اولیه‌ی شما هر ۲۰ms مقدار را بدون شرط می‌فرستاد و کار هم می‌کرد؛ نسخه‌ی بالا بهینه‌تر است چون صف را پر نمی‌کند و ترافیک غیرضروری ندارد. در حالت ارسال دائمی، خواندن با `while` در Model باعث می‌شود صف خالی شود و مشکلی پیش نیاید.

۴.۲ تابع HAL_GPIO_ReadPin

آرگومان	معنی
`GPIO_TypeDef *GPIOx`	پورت، مثلاً `GPIOH`. ماکروی `KEY1_GPIO_Port` از User Label ساخته می‌شود.
`uint16_t GPIO_Pin`	ماسک پایه، مثلاً `GPIO_PIN_x`. ماکروی `KEY1_Pin`.
بازگشتی `GPIO_PinState`	`GPIO_PIN_SET` (=1) یا `GPIO_PIN_RESET` (=0).

مدار کلید مهم است: اگر کلید به زمین وصل می‌شود و Pull-up دارد، «فشرده» = `0`. اگر به VCC وصل می‌شود و Pull-down دارد، «فشرده» = `1`. در `MX_GPIO_Init` شما برای KEY1 حالت `GPIO_NOPULL` است؛ پس مدار برد باید خودش Pull داشته باشد، وگرنه ورودی شناور و مقدار تصادفی می‌شود. چون مثال شما درست کار کرد، مدار بردتان با منطق `state != 0 ⇒ Pressed` سازگار است.

۴.۳ تابع osMessageQueuePut

```
osStatus_t osMessageQueuePut(osMessageQueueId_t mq_id,
                             const void *msg_ptr,
                             uint8_t msg_prio,
                             uint32_t timeout);
```

آرگومان	معنی
`mq_id`	شناسه‌ی صف؛ در اینجا `QueueButtonHandle`.
`msg_ptr`	آدرس داده‌ای که باید در صف کپی شود (`&state`). به اندازه‌ی `msg_size` بایت از آن کپی می‌شود.
`msg_prio`	اولویت پیام؛ در FreeRTOS نادیده گرفته می‌شود (FIFO ساده)، معمولاً 0.
`timeout`	مدت انتظار (تیک) اگر صف پر باشد. `0` = بدون انتظار و بازگشت فوری، `osWaitForever` = تا ابد صبر.

مقدار بازگشتی: `osOK` موفق؛ `osErrorResource` صف پر است (با timeout=0)؛ `osErrorTimeout` پایان مهلت؛ `osErrorParameter` آرگومان نامعتبر. در کد حرفه‌ای مقدار بازگشتی را بررسی کنید.

۴.۴ تابع osDelay

`osStatus_t osDelay(uint32_t ticks)`: تسک را برای `ticks` تیک سیستم به حالت Blocked می‌برد تا CPU به تسک‌های دیگر برسد. اگر `configTICK_RATE_HZ = 1000` باشد، هر تیک = ۱ms و `osDelay(20)` = ۲۰ms. بدون osDelay، تسک CPU را می‌بلعد (در اولویت یکسان، تسک GUI گرسنه می‌ماند).

چرا ۲۰ms؟ این فاصله، یک *نمونه‌برداری و debounce* ساده را هم انجام می‌دهد: جهش‌های مکانیکی چند میلی‌ثانیه‌ای کلید بین دو نمونه از بین می‌روند و تقریباً ثبت نمی‌شوند.

۴.۵ چرا فقط هنگام تغییر؟ (مقایسه)

روش	مزیت	عیب
ارسال هر ۲۰ms بدون شرط	ساده	ترافیک ۵۰ پیام در ثانیه، احتمال پر شدن صف، مصرف CPU بیهوده، ارسال داده‌ی تکراری
ارسال فقط هنگام تغییر	کم‌مصرف، مناسب رویداد	اگر صفحه‌ای که باز می‌شود وضعیت فعلی را لازم داشته باشد، باید آن را در Model نگه دارید (ذخیره‌ی آخرین مقدار)

۵. Model.hpp و Model.cpp

۵.۱ Model.hpp

```
#ifndef MODEL_HPP
#define MODEL_HPP

class ModelListener;          /* forward declaration */

class Model
{
public:
    Model();
    void bind(ModelListener* listener) { modelListener = listener; }
    void tick();
protected:
    ModelListener* modelListener;
};
#endif
```

- **Forward declaration** (`class ModelListener;`): چون `ModelListener.hpp` خودش `Model.hpp` را include می‌کند، include کردن متقابل حلقه ایجاد می‌کند. فقط اعلام نام کلاس کافی است چون در Model فقط اشاره‌گر نگه می‌داریم.
- **`bind()`** : فریم‌ورک هنگام ساخت صفحه آن را صدا می‌زند و Presenter را به‌عنوان listener ثبت می‌کند.
- **`tick()`** : توسط TouchGFX در هر فریم صدا زده می‌شود.

۵.۲ Model.cpp (نسخه‌ی صحیح)

```
#include <gui/model/Model.hpp>
#include <gui/model/ModelListener.hpp>

#ifndef SIMULATOR
#include "stdbool.h"
#include "stdint.h"
#include "cmsis_os2.h"
extern "C" osMessageQueueId_t QueueButtonHandle;
#endif

Model::Model() : modelListener(0) {}

void Model::tick()
{
#ifndef SIMULATOR
    uint32_t state;
    while (osMessageQueueGet(QueueButtonHandle, &state, NULL, 0) == osOK)
    {
        bool buttonPressed = (state != 0);
        if (modelListener)
            modelListener->notifyButtonChanged(buttonPressed);
    }
#endif
}
```

۵.۳ تابع osMessageQueueGet

```
osStatus_t osMessageQueueGet(osMessageQueueId_t mq_id,
                             void *msg_ptr,
                             uint8_t *msg_prio,
                             uint32_t timeout);
```

آرگومان	معنی
`mq_id`	شناسه‌ی صف. **باید `QueueButtonHandle` باشد.** (در نسخه‌ی اول شما اشتباهاً `modelListener` بود: یک اشاره‌گر C++ که صف نیست؛ چون نوع `osMessageQueueId_t` در واقع `void*` است، کامپایلر هیچ خطایی نداد!)
`msg_ptr`	آدرس متغیری که پیام در آن کپی می‌شود (`&state`). باید حداقل `msg_size` بایت جا داشته باشد.
`msg_prio`	اشاره‌گر برای دریافت اولویت پیام؛ `NULL` یعنی نیازی نیست.
`timeout`	**در تسک GUI باید 0 باشد.** اگر مثلاً `osWaitForever` بگذارید، تسک GUI تا رسیدن پیام قفل می‌شود و صفحه فریز می‌شود.

بازگشتی: `osOK` یعنی پیامی دریافت شد؛ `osErrorResource` یعنی صف خالی است (با timeout=0)؛ `osErrorTimeout` پایان مهلت.

۵.۴ نکته‌های مهم

- **`extern "C"`:** متغیر در `main.c` (زبان C) تعریف شده؛ در C++ نام‌ها mangle می‌شوند. برای اینکه لینکر همان متغیر C را پیدا کند، اعلان را داخل `extern "C"` بگذارید. این خط در `#ifndef SIMULATOR` است چون Simulator روی PC اجرا می‌شود و `main.c` و FreeRTOS ندارد.
- **`while` به‌جای `if`:** همه‌ی پیام‌های جمع‌شده در یک tick خوانده و صف خالی می‌شود. با `if` فقط یک پیام در هر فریم مصرف می‌شود و اگر سرعت تولید از ۶۰Hz بیشتر باشد، صف عقب می‌ماند.
- **تطابق نوع:** نوع `state` در `uint32_t` (Model) باید با نوع ارسالی و Item Size صف یکی باشد.
- **Simulator:** در شبیه‌ساز، `tick()` خالی است؛ برای تست می‌توانید داخل `#ifdef SIMULATOR` مقدار تصادفی یا زمان‌دار تولید کنید تا UI را بدون سخت‌افزار آزمایش کنید.

۶. ModelListener و screenPresenter

۶.۱ ModelListener.hpp

```
class ModelListener
{
public:
    ModelListener() : model(0) {}
    virtual ~ModelListener() {}

    void bind(Model* m) { model = m; }

    virtual void notifyButtonChanged(bool buttonPressed) {}

protected:
    Model* model;
};
```

- **`virtual notifyButtonChanged`:** تابع «اعلان». بدنه‌ی خالی دارد تا Presenterهایی که به آن نیاز ندارند ملزم به پیاده‌سازی نباشند.
- destructor: **`virtual ~ModelListener()`** مجازی برای نابودی درست شیء مشتق‌شده از طریق اشاره‌گر پایه ضروری است.
- **`Model* model`** و Presenter: **`bind(Model*)`** از طریق این اشاره‌گر می‌تواند به Model فرمان بدهد (مثلاً `model->setLed(true)`). فریم‌ورک آن را در ساخت صفحه صدا می‌زند.
- برای هر داده‌ی جدید، یک تابع virtual جدید اینجا اضافه می‌کنید.

۶.۲ screenPresenter.hpp و .cpp

```
class screenPresenter : public touchgfx::Presenter, public ModelListener
{
public:
    screenPresenter(screenView& v);
    virtual void activate();
    virtual void deactivate();
    virtual ~screenPresenter() {}

    virtual void notifyButtonChanged(bool pressed) override;  /* نام دقیقاً مثل پایه ! */
private:
    screenPresenter();
    screenView& view;
};

void screenPresenter::notifyButtonChanged(bool pressed)
{
    view.updateButtonState(pressed);
}
```

عنصر	توضیح
`touchgfx::Presenter`	کلاس پایه‌ی فریم‌ورک؛ تعریف‌کننده‌ی `activate()` و `deactivate()`.
`public ModelListener`	ارث‌بری چندگانه: Presenter هم «Presenter» است هم «شنونده‌ی Model».
`screenView& view`	ارجاع به View که در سازنده گرفته می‌شود. چون ارجاع است باید در initializer list مقداردهی شود: `: view(v)`.
`private: screenPresenter();`	سازنده‌ی پیش‌فرض عمداً private است تا کسی Presenter بدون View نسازد.
`class screenView; (forward)`	در hpp فقط نام کلاس View کافی است؛ `screenView.hpp` در cpp include می‌شود.

باگ اصلی پروژه‌ی شما: در Presenter نام تابع `notiyButtonChanged` (بدون حرف f) نوشته شده بود. در C++ این یک تابع جدید است و تابع پایه را override نمی‌کند، پس Model همیشه نسخه‌ی خالی ModelListener را صدا می‌زد. هیچ خطا یا هشداری هم داده نمی‌شود. راه جلوگیری: همیشه کلمه‌ی `override` را بنویسید؛ آن‌وقت اگر تابعی در کلاس پایه با همان امضا وجود نداشته باشد، کامپایلر خطا می‌دهد.

۶.۳ چه کارهایی در Presenter مجاز است؟

- تبدیل داده‌ی خام Model به شکل مناسب نمایش (مثلاً `state != 0` به `bool`، یا ADC خام به درصد).
- تصمیم‌گیری درباره‌ی نمایش (مثلاً اگر دما > ۸۰ هشدار نشان بده).
- انتقال فرمان‌های کاربر به Model: `model->xyz()`.
- **مجاز نیست:** دسترسی مستقیم به ویجت‌ها (مثل `textArea1`)؛ آن‌ها فقط در View هستند.

> اصل ساده: «Presenter می‌گوید **چه** نشان بده؛ View می‌داند **چگونه** نشان بدهد.»

۷. screenView و ویجت‌ها

۷.۱ screenView.hpp / .cpp

```
class screenView : public screenViewBase
{
public:
    screenView();
    virtual ~screenView() {}
    virtual void setupScreen();
    virtual void tearDownScreen();
    void updateButtonState(bool pressed);      /* تابع ما */
};

void screenView::updateButtonState(bool pressed)
{
    if (pressed)
        Unicode::strncpy(textArea1Buffer, "Pressed", TEXTAREA1_SIZE);
    else
        Unicode::strncpy(textArea1Buffer, "Not Pressed", TEXTAREA1_SIZE);

    textArea1.invalidate();   /* درخواست بازرسم ناحیه‌ی ویجت */
}
```

۷.۲ ساخت متن پویا (Wildcard) در Designer

1. یک **Text Area** روی صفحه بگذارید و نام آن را `textArea1` کنید.
2. در بخش Texts، متن آن را به‌صورت (Wildcard) `<placeholder>` تعریف کنید؛ مثلاً در ورودی متن بنویسید `<Not Pressed>` یا یک رشته‌ی دارای `<>`.
3. در پنجره‌ی **Wildcard Buffer Size**، Wildcard را حداقل برابر طولانی‌ترین متن + ۱ بگذارید (برای «Not Pressed» حداقل ۱۲).
4. Generate Code. در `screenViewBase.hpp` این‌ها ساخته می‌شوند:
   `touchgfx::Unicode::UnicodeChar textArea1Buffer[TEXTAREA1_SIZE];` و ماکروی `TEXTAREA1_SIZE`.
   `touchgfx::TextAreaWithOneWildcard textArea1;`
5. عرض خود ویجت را به اندازه‌ای بزرگ بگیرید که طولانی‌ترین متن جا شود؛ در غیر این صورت متن بریده می‌شود.

> TouchGFX برای رندر، متن‌ها را به‌صورت `UnicodeChar` (۱۶ بیتی) ذخیره می‌کند، نه `char`. به همین دلیل نمی‌توان با `strcpy` معمولی متن را کپی کرد و باید از کلاس `Unicode` استفاده کرد.

۷.۳ توابع کلاس Unicode و ویجت

تابع	شرح و آرگومان‌ها
`Unicode::strncpy(dst, src, n)`	`dst`: بافر `UnicodeChar` مقصد؛ `src`: رشته‌ی C معمولی (`const char*`)؛ `n`: حداکثر تعداد کاراکتر شامل پایان‌دهنده، یعنی اندازه‌ی بافر (`TEXTAREA1_SIZE`). برای رشته‌ی طولانی‌تر، بریده می‌شود.
`Unicode::snprintf(dst, n, fmt, ...)`	مثل snprintf با فرمت `"%d"` و `"%s"`؛ برای نمایش عدد: `Unicode::snprintf(textArea1Buffer, TEXTAREA1_SIZE, "%d", value);`. فرمت با `<placeholder>` داخل متن Designer ترکیب می‌شود.
`widget.invalidate()`	ناحیه‌ی ویجت را «کثیف» علامت می‌زند تا در فریم بعد بازرسم شود. بدون آن، بافر عوض می‌شود ولی روی LCD تغییری دیده نمی‌شود.
`widget.resizeToCurrentText()`	اندازه‌ی ویجت را با طول متن جدید تنظیم می‌کند (برای متن‌های با طول متغیر؛ قبل از invalidate).
`screenViewBase::setupScreen()`	در override حتماً صدا بزنید؛ ویجت‌های تولیدشده را مقداردهی می‌کند.

۷.۴ ارتباط View → Presenter (برای دکمه‌ها)

اگر کاربر دکمه‌ی لمسی روی صفحه را فشار دهد، در Designer یک Interaction تعریف می‌کنید: «Button clicked → Call new virtual function `buttonClicked()`». Designer آن را در `screenView.cpp` شما می‌سازد؛ این تابع virtual را در `screenViewBase` پیاده‌سازی و از طریق `presenter->onButton()` به Presenter و سپس `model->sendToTask()` به Model می‌دهید، که داخلش `osMessageQueuePut` روی یک صف دیگر (از GUI به تسک سخت‌افزار) انجام می‌شود. در این جهت، فرستنده GUI و گیرنده تسک سخت‌افزار است و برای گیرنده معمولاً `osWaitForever` می‌گذارید (چون تسک خودش مسدود می‌شود، نه GUI).

۸. جریان کامل، خطاهای رایج و الگوی تعمیم

۸.۱ جریان دقیق رویداد (به ترتیب زمان)

1. کاربر کلید را فشار می‌دهد؛ پایه‌ی `KEY1` تغییر سطح می‌دهد.
2. حداکثر ۲۰ms بعد، `StartButtonTask` بیدار می‌شود، `HAL_GPIO_ReadPin` را می‌خواند و چون با آخرین مقدار فرق دارد، `osMessageQueuePut` پیام را در `QueueButton` کپی می‌کند.
3. در فریم بعدی TouchGFX (≥۱۶٫۷ms)، `Model::tick()` اجرا می‌شود و `osMessageQueueGet` پیام را بیرون می‌کشد.
4. Model چون `modelListener` غیر تهی است، `notifyButtonChanged(bool)` را صدا می‌زند (فراخوانی مجازی ⇒ نسخه‌ی `screenPresenter`).
5. `view.updateButtonState(pressed)`، Presenter را فرا می‌خواند.
6. View متن را در بافر Wildcard می‌نویسد و `invalidate()` می‌زند.
7. در رندر فریم بعد، ناحیه‌ی بازرسم و از طریق LTDC (بافر در SDRAM، آدرس `0xC0000000`) روی LCD نمایش داده می‌شود.

تأخیر کل معمولاً کمتر از ~۴۰ms است، یعنی برای کاربر آنی.

۸.۲ جدول خطاهایی که در پروژه‌ی شما پیش آمد

خطا	علامت	راه‌حل
غلط املایی تابع (`notiy...`)	بدون خطای کامپایل؛ هیچ تغییری روی LCD	نام یکسان + استفاده از `override`
اشتباه در Get handle (`modelListener`)	همیشه شکست؛ بدون خطای کامپایل	استفاده از `QueueButtonHandle`
Item Size ≠ نوع داده (۲ در برابر ۴ بایت)	مقدار خراب یا ناقص	یکسان‌سازی در CubeMX
نبودن `extern "C"`	خطای لینکر undefined reference	اعلان `extern "C"`
Wildcard Buffer کوچک	متن بریده	افزایش اندازه‌ی بافر و عرض ویجت
فراموش کردن `invalidate()`	بافر عوض ولی صفحه بدون تغییر	فراخوانی بعد از تغییر

۸.۳ الگوی ثابت برای هر مثال جدید (قالب)

فرض کنید می‌خواهید مقدار ADC را روی یک Progress Bar نشان دهید. همان هشت مرحله:

1. **CubeMX:** صف `QueueAdc`، اندازه‌ی آیتم `uint16_t`؛ تسک `AdcTask`.
2. **تسک:** خواندن ADC و `osMessageQueuePut(QueueAdcHandle, &value, 0, 0)`.
3. **Model.cpp:** `extern "C" osMessageQueueId_t QueueAdcHandle;` و در `tick()`: `osMessageQueueGet(QueueAdcHandle, &v, NULL, 0)` سپس `modelListener->notifyAdcChanged(v)`.
4. **ModelListener.hpp:** `virtual void notifyAdcChanged(uint16_t v) {}`.
5. **Presenter:** `void notifyAdcChanged(uint16_t v) override { view.updateAdc(v); }`.
6. **View:** `void updateAdc(uint16_t v) { progress1.setValue(v); progress1.invalidate(); }`.
7. **Designer:** ویجت‌ها را بگذارید و Generate Code بزنید.
8. **تست:** ابتدا در Simulator با داده‌ی ساختگی، بعد روی برد.

> قاعده‌ی حفظ‌کردنی: «هر نوع داده = یک صف (یا یک struct واحد) + یک تابع notify در ModelListener + یک override در Presenter + یک تابع update در View».

۸.۴ چک‌لیست عیب‌یابی سریع

- اول بررسی کنید handle صف NULL نباشد (breakpoint بعد از `osMessageQueueNew`).
- در breakpoint `tick()` بگذارید: آیا `osMessageQueueGet` مقدار `osOK` می‌دهد؟
- آیا `modelListener` تهی نیست؟ آیا تابع Presenter صدا می‌خورد؟
- آیا `textArea1` روی صفحه دیده می‌شود (رنگ متن با پس‌زمینه یکی نباشد)؟
- پشته‌ی تسک‌ها را با `uxTaskGetStackHighWaterMark` بسنجید؛ تسک GUI در پروژه‌ی شما ۸ کیلوبایت دارد.

منابع و مراجع برای مطالعه‌ی بیشتر

- مستندات رسمی TouchGFX (`support.touchgfx.com`): بخش‌های Model-View-Presenter Design Pattern، Wildcard Text و Resources و Backend Communication.
- مستندات CMSIS-RTOS2 (`arm-software.github.io/CMSIS_5/RTOS2`): فصل Message Queue (توابع `osMessageQueueNew/Put/Get`) و Generic Wait Functions (`osDelay`).
- مستندات FreeRTOS (`freertos.org`): بخش Queue Management و Task Management.
- راهنمای ST HAL (UM2217) برای خانواده‌ی STM32H7 برای `HAL_GPIO_ReadPin`.
- سورس‌های خود پروژه: `FrontendApplicationBase.cpp`، `main.c` (محل ساخت و bind کردن Presenter و Model)، `screenViewBase.hpp`.

توجه: این سند بر پایه‌ی دانش عمومی از معماری TouchGFX/CMSIS-RTOS2 و کدهای شما نوشته شده است؛ برای جزئیات دقیق نسخه‌ی خودتان (مثلاً رفتار دقیق `Unicode::strncpy`)، به مستندات نسخه‌ی TouchGFX نصب‌شده مراجعه کنید.